



The University of Azad Jammu and Kashmir

Name: Zarnab Fatima

Roll NO: 2024-SE-14

Semester:4th

Course Title: Database System

Course Code: CS-2204

Submitted To: Engr. Awais Rathore

Department of Software Engineering

LAB 10

1. Introduction

Scalar functions operate on one row at a time and return one value for each row. In this lab, string functions are used to clean, format, extract, and modify text data stored in the Customer and Product tables.

The **main** string functions used in this lab are:

- TRIM()
- UPPER()
- LOWER()
- CHAR_LENGTH()
- CONCAT()
- SUBSTRING()
- LEFT()
- REPLACE()
- LPAD()
- LOCATE()

1.Create Database

```
CREATE DATABASE IF NOT EXISTS  
scalar_lab;  
USE scalar_lab;
```



2. Create and Insert Tables (Customer , Product)

Table	Action	Rows	Type	Collation	Size	Overhead
☐ customer		0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ product		0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
2 tables Sum		0	InnoDB	utf8mb4_general_ci	32.0 KiB	0 B

`SELECT * FROM `customer``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

		CustID	CustName	Email	City	Phone	JoinDate	DOB
☐		1	Ali Khan	ali.khan@MAIL.com	Lahore	0300-1112233	2022-01-15	1995-04-12
☐		2	Sara Iqbal	sara@example.com	Karachi	0301-4445566	2022-04-22	1998-11-20
☐		3	HAMZA RAZA	hamza@example.com	Lahore	0302-7778899	2023-02-10	1997-08-05
☐		4	Ayesha Noor	NULL	Islamabad	0303-1234567	2023-05-18	1999-02-14
☐		5	bilal ahmed	bilal@MAIL.COM	Karachi	0304-2345678	2023-09-01	2000-06-30
☐		6	Fatima Sheikh	fatima@example.com	NULL	0305-3456789	2024-01-12	1996-10-25
☐		7	Usman Tariq	usman@example.com	Lahore	NULL	2024-06-30	2001-03-18
☐		8	Maira Javed	maira@example.com	Islamabad	0307-5678901	2024-08-25	1994-12-09

SELECT * FROM `product`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		ProdID	ProdName	Category	Price	StockQty	LaunchDate
☐	Edit Copy Delete	101	Laptop Pro 15	Electronics	185000.00	12	2023-03-10
☐	Edit Copy Delete	102	Wireless Mouse	Electronics	2500.00	50	2022-07-22
☐	Edit Copy Delete	103	USB-C Cable	Electronics	800.00	100	2021-11-05
☐	Edit Copy Delete	104	Office Chair	Furniture	18500.00	8	2023-01-15
☐	Edit Copy Delete	105	Standing Desk	Furniture	45000.50	5	2024-02-28
☐	Edit Copy Delete	106	Notebook A4	Stationery	350.00	200	2020-04-01
☐	Edit Copy Delete	107	Ballpoint Pen 10pk	Stationery	450.00	150	2020-04-01
☐	Edit Copy Delete	108	Coffee Beans 1kg	Grocery	1899.99	30	2023-09-20
☐	Edit Copy Delete	109	Green Tea Box	Grocery	650.00	45	2022-12-12
☐	Edit Copy Delete	110	Bluetooth Speaker	Electronics	7500.00	18	2024-05-18

2. Part A — String Functions

Task A1 — Remove Leading and Trailing Spaces

Objective:

List each customer's name with leading and trailing spaces removed while also displaying the original name.

SQL Query:

```
SELECT  CustID,
        CustName AS OriginalName,
        TRIM(CustName) AS CleanedName
FROM Customer;
```

Functions Used: TRIM()

Description: The TRIM() function removes unnecessary spaces from the beginning and end of the customer's name.

✓ Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)

```
SELECT CustID, CustName AS OriginalName, TRIM(CustName) AS CleanedName FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by ke

Extra options

				CustID	OriginalName	CleanedName
☐		Edit		Copy		Delete
1	Ali Khan	Ali Khan				
2	Sara Iqbal	Sara Iqbal				
3	HAMZA RAZA	HAMZA RAZA				
4	Ayesha Noor	Ayesha Noor				
5	bilal ahmed	bilal ahmed				
6	Fatima Sheikh	Fatima Sheikh				
7	Usman Tariq	Usman Tariq				
8	Maira Javed	Maira Javed				

Task A2 — Convert Names to Uppercase and Lowercase

Objective:

Display every customer's name in uppercase and lowercase.

SQL Query:

```
SELECT CustID,
       UPPER(CustName) AS
       UpperName,
       LOWER(CustName) AS
       LowerName FROM Customer;
```

Functions Used: UPPER(), LOWER()

Description: UPPER() converts all letters to uppercase, while LOWER() converts all letters to lowercase.

`SELECT CustID, UPPER(CustName) AS UpperName, LOWER(CustName) AS LowerName FROM Customer;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ | Filter rows: | Sort by key:

Extra options

					CustID	UpperName	LowerName
☐		Edit		Copy		Delete	1 ALI KHAN ali khan
☐		Edit		Copy		Delete	2 SARA IQBAL sara iqbal
☐		Edit		Copy		Delete	3 HAMZA RAZA hamza raza
☐		Edit		Copy		Delete	4 AYESHA NOOR ayesha noor
☐		Edit		Copy		Delete	5 BILAL AHMED bilal ahmed
☐		Edit		Copy		Delete	6 FATIMA SHEIKH fatima sheikh
☐		Edit		Copy		Delete	7 USMAN TARIQ usman tariq
☐		Edit		Copy		Delete	8 MAIRA JAVED maira javed

Task A3 — Trim Names and Count Characters

Objective:

Display each customer's trimmed name and the number of characters in it.

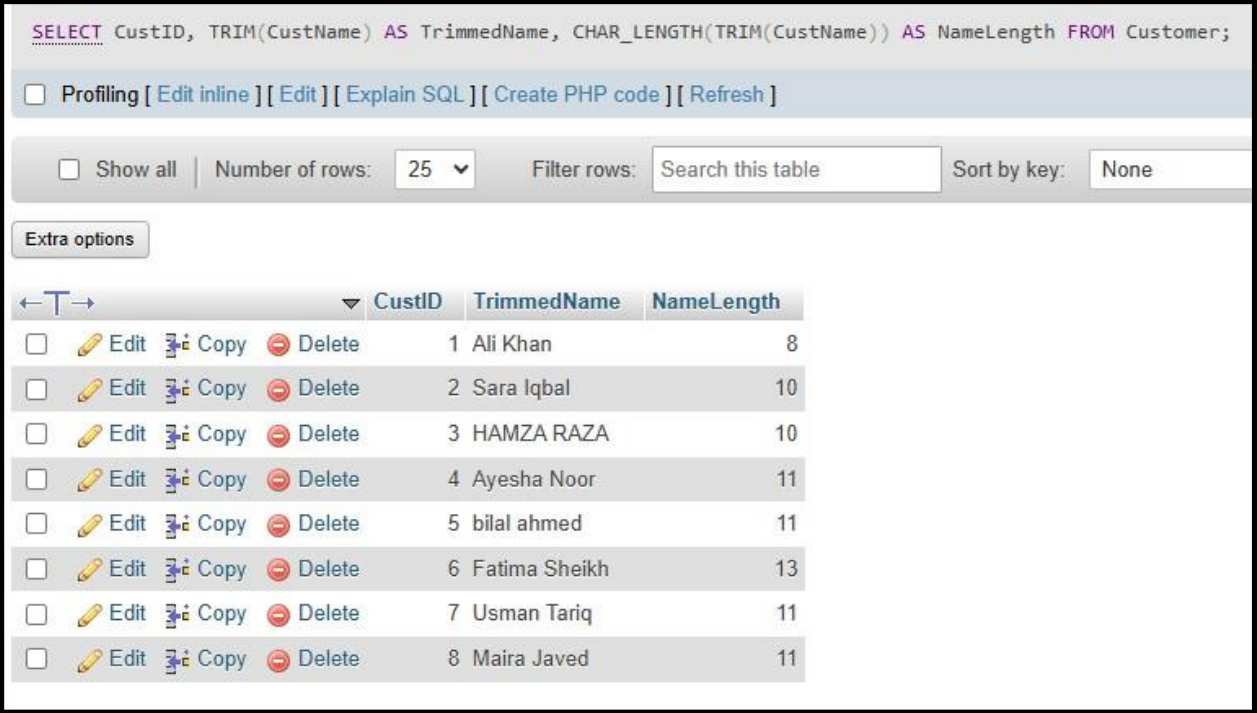
SQL Query:

```
SELECT  CustID,
        TRIM(CustName) AS
TrimmedName,

        CHAR_LENGTH(TRIM(CustName))
AS NameLength FROM Customer;
```

Functions Used: TRIM(), CHAR_LENGTH()

Description: First, TRIM() removes unnecessary spaces. Then CHAR_LENGTH() counts the characters in the cleaned name.



```
SELECT CustID, TRIM(CustName) AS TrimmedName, CHAR_LENGTH(TRIM(CustName)) AS NameLength FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				CustID	TrimmedName	NameLength
☐				1	Ali Khan	8
☐				2	Sara Iqbal	10
☐				3	HAMZA RAZA	10
☐				4	Ayesha Noor	11
☐				5	bilal ahmed	11
☐				6	Fatima Sheikh	13
☐				7	Usman Tariq	11
☐				8	Maira Javed	11

Task A4 — Create a Customer Greeting

Objective:

Build a greeting for every customer using their trimmed name.

SQL Query:

```
SELECT CustID,  
CONCAT('Dear ',  
TRIM(CustName), ', welcome!')  
AS  
Greeting FROM Customer;
```

Functions Used: CONCAT(), TRIM()

Description: CONCAT() combines multiple pieces of text into one string.

Example Output:

Dear Ali Khan, welcome!

```
SELECT CustID, CONCAT('Dear ', TRIM(CustName), ', welcome!') AS Greeting FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ | Filter rows: | Sort by key:

Extra options

	CustID	Greeting
☐ Edit Copy Delete	1	Dear Ali Khan, welcome!
☐ Edit Copy Delete	2	Dear Sara Iqbal, welcome!
☐ Edit Copy Delete	3	Dear HAMZA RAZA, welcome!
☐ Edit Copy Delete	4	Dear Ayesha Noor, welcome!
☐ Edit Copy Delete	5	Dear bilal ahmed, welcome!
☐ Edit Copy Delete	6	Dear Fatima Sheikh, welcome!
☐ Edit Copy Delete	7	Dear Usman Tariq, welcome!
☐ Edit Copy Delete	8	Dear Maira Javed, welcome!

Task A5 — Extract Username from Email

Objective:

For customers who have an email address, extract the part before the @ symbol.

SQL Query:

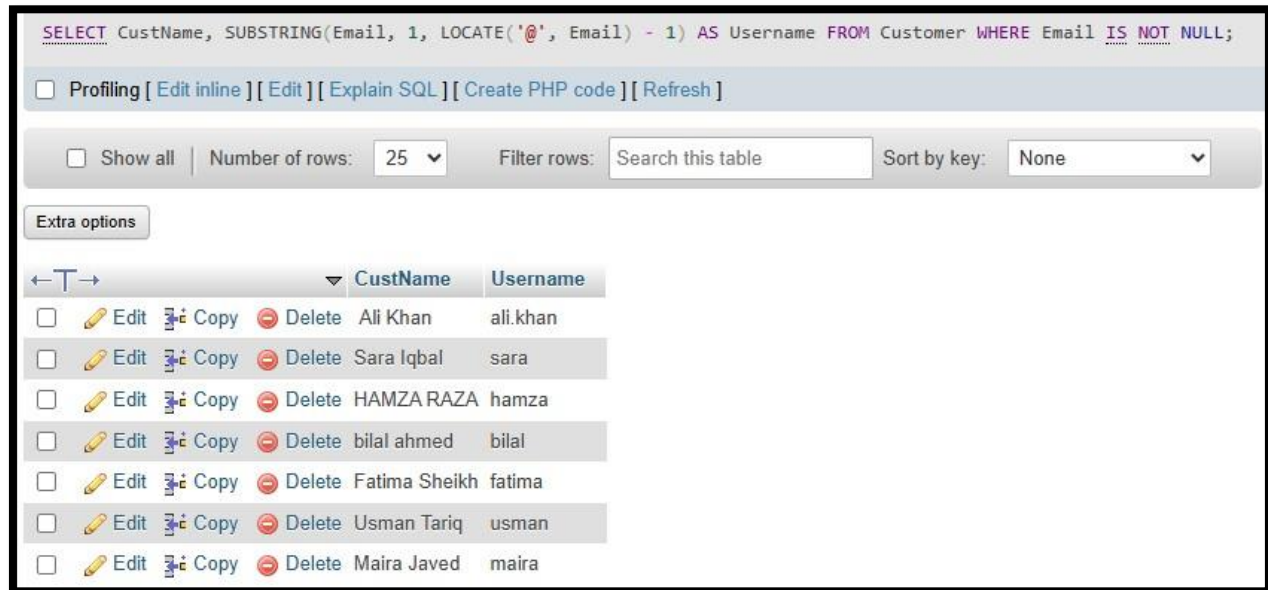
```
SELECT CustName,
       SUBSTRING(Email, 1,
       LOCATE('@', Email) - 1) AS
       Username FROM Customer
WHERE Email IS NOT NULL;
```

Functions Used: SUBSTRING(), LOCATE()

Description: LOCATE() finds the position of @, and SUBSTRING() extracts everything before it.

Example: ali.khan@MAIL.com

Result: ali.khan



`SELECT CustName, SUBSTRING(Email, 1, LOCATE('@', Email) - 1) AS Username FROM Customer WHERE Email IS NOT NULL;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				CustName	Username
☐	Edit	Copy	Delete	Ali Khan	ali.khan
☐	Edit	Copy	Delete	Sara Iqbal	sara
☐	Edit	Copy	Delete	HAMZA RAZA	hamza
☐	Edit	Copy	Delete	bilal ahmed	bilal
☐	Edit	Copy	Delete	Fatima Sheikh	fatima
☐	Edit	Copy	Delete	Usman Tariq	usman
☐	Edit	Copy	Delete	Maira Javed	maira

Task A6 — Extract Email Domain

Objective:

For customers who have an email address, extract everything after the @ symbol.

SQL Query:

```
SELECT
    CustName,
    SUBSTRING(Email,
LOCATE('@', Email) + 1) AS
Domain
FROM Customer
WHERE Email IS NOT NULL;
```

Functions Used: SUBSTRING(), LOCATE()

Description: The query finds the @ symbol and extracts the text after it.

Example: ali.khan@MAIL.com

Result: MAIL.com

```
SELECT CustName, SUBSTRING(Email, LOCATE('@', Email) + 1) AS Domain FROM Customer WHERE Email IS NOT NULL;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

					CustName	Domain
☐					Ali Khan	MAIL.com
☐					Sara Iqbal	example.com
☐					HAMZA RAZA	example.com
☐					bilal ahmed	MAIL.COM
☐					Fatima Sheikh	example.com
☐					Usman Tariq	example.com
☐					Maira Javed	example.com

Task A7 — Display First Three Characters

Objective:

Show only the first three characters of each customer's name.

SQL Query:

```
SELECT CustID,
       LEFT(TRIM(CustName), 3)
AS
First3Characters FROM
Customer;
```

Functions Used: LEFT(), TRIM()

Description: LEFT() returns the specified number of characters from the beginning of a string.

✓ Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)

```
SELECT CustID, LEFT(TRIM(CustName), 3) AS First3Characters FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by

Extra options

					CustID	First3Characters		
☐		Edit		Copy		Delete	1	Ali
☐		Edit		Copy		Delete	2	Sar
☐		Edit		Copy		Delete	3	HAM
☐		Edit		Copy		Delete	4	Aye
☐		Edit		Copy		Delete	5	bil
☐		Edit		Copy		Delete	6	Fat
☐		Edit		Copy		Delete	7	Usm
☐		Edit		Copy		Delete	8	Mai

Task A8 — Mask Customer Phone Numbers

Objective:

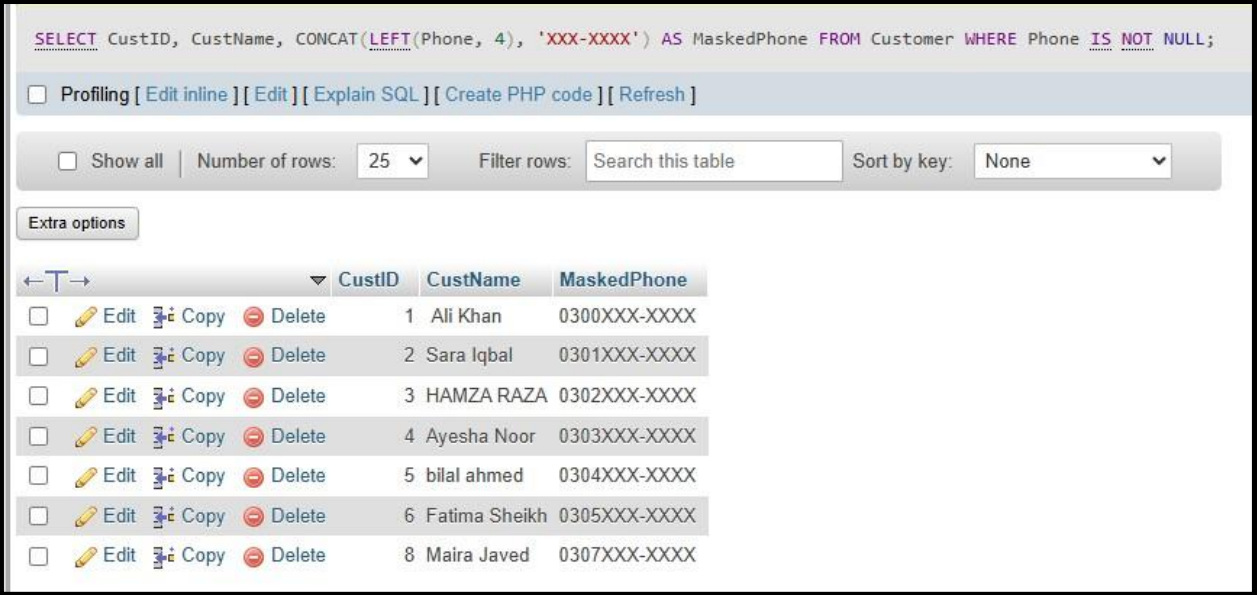
Display the first four characters of each phone number followed by XXX-XXXX. Customers without phone numbers should be skipped.

SQL Query:

```
SELECT CustID, CustName,
       CONCAT(LEFT(Phone, 4),
              'XXX-XXXX') AS MaskedPhone
FROM Customer
WHERE Phone IS NOT NULL;
```

Functions Used: LEFT(), CONCAT()

Description: The first four characters of the phone number are displayed, while the remaining portion is replaced with XXX-XXXX.



The screenshot shows a database query editor with the following SQL query:

```
SELECT CustID, CustName, CONCAT(LEFT(Phone, 4), 'XXX-XXXX') AS MaskedPhone FROM Customer WHERE Phone IS NOT NULL;
```

Below the query, there are controls for Profiling, Show all, Number of rows (25), Filter rows (Search this table), and Sort by key (None). An "Extra options" button is also present.

The results are displayed in a table with the following columns: CustID, CustName, and MaskedPhone. Each row includes checkboxes for Edit, Copy, and Delete actions.

	CustID	CustName	MaskedPhone
☐ Edit ☐ Copy ☐ Delete	1	Ali Khan	0300XXX-XXXX
☐ Edit ☐ Copy ☐ Delete	2	Sara Iqbal	0301XXX-XXXX
☐ Edit ☐ Copy ☐ Delete	3	HAMZA RAZA	0302XXX-XXXX
☐ Edit ☐ Copy ☐ Delete	4	Ayesha Noor	0303XXX-XXXX
☐ Edit ☐ Copy ☐ Delete	5	bilal ahmed	0304XXX-XXXX
☐ Edit ☐ Copy ☐ Delete	6	Fatima Sheikh	0305XXX-XXXX
☐ Edit ☐ Copy ☐ Delete	8	Maira Javed	0307XXX-XXXX

Task A9 — Replace Spaces with Hyphens

Objective:

Replace all spaces in product names with hyphens.

SQL Query:

```
SELECT ProdID,  
       REPLACE(ProdName, ' ', '-')  
AS SlugName FROM Product;
```

Function Used: REPLACE()

Description: REPLACE() replaces every occurrence of one character or string with another.

Example: Laptop Pro 15

becomes:

Laptop-Pro-15

✓ Showing rows 0 - 9 (10 total, Query took 0.0004 seconds.)

```
SELECT ProdID, REPLACE(ProdName, ' ', '-') AS SlugName FROM Product;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ | Filter rows: | Sort

Extra options

	ProdID	SlugName
☐ Edit Copy Delete	101	Laptop-Pro-15
☐ Edit Copy Delete	102	Wireless-Mouse
☐ Edit Copy Delete	103	USB-C-Cable
☐ Edit Copy Delete	104	Office-Chair
☐ Edit Copy Delete	105	Standing-Desk
☐ Edit Copy Delete	106	Notebook-A4
☐ Edit Copy Delete	107	Ballpoint-Pen-10pk
☐ Edit Copy Delete	108	Coffee-Beans-1kg
☐ Edit Copy Delete	109	Green-Tea-Box
☐ Edit Copy Delete	110	Bluetooth-Speaker

☒ Console

Task A10 — Pad Product IDs with Zeros

Objective:

Display each product ID as a five-digit value by adding leading zeros.

SQL Query:

```
SELECT
    ProdID,
    LPAD(ProdID, 5, '0') AS
PaddedProdID
FROM Product;
```

Function Used: LPAD()

Description: LPAD() adds characters to the left side of a value until it reaches the specified length.

Example: 101 → 00101

The screenshot shows a SQL query execution interface. At the top, the query is: `SELECT ProdID, LPAD(ProdID, 5, '0') AS PaddedProdID FROM Product;`. Below the query, there are options for **Profiling**, **Edit inline**, **Edit**, **Explain SQL**, **Create PHP code**, and **Refresh**. Further down, there are controls for **Show all**, **Number of rows** (set to 25), and a **Filter rows** search box. An **Extra options** button is also present. The main area displays a table with two columns: **ProdID** and **PaddedProdID**. The table contains 10 rows of data, with ProdID values from 101 to 110 and corresponding PaddedProdID values with leading zeros. Each row has interactive icons for **Edit**, **Copy**, and **Delete**. At the bottom left, there is a **Console** tab.

	ProdID	PaddedProdID
☐ Edit Copy Delete	101	00101
☐ Edit Copy Delete	102	00102
☐ Edit Copy Delete	103	00103
☐ Edit Copy Delete	104	00104
☐ Edit Copy Delete	105	00105
☐ Edit Copy Delete	106	00106
☐ Edit Copy Delete	107	00107
☐ Edit Copy Delete	108	00108
☐ Edit Copy Delete	109	00109
☐ Edit Copy Delete	110	00110

Task A11 — Find Products Containing "Pro"

Objective:

Display product names that contain the word Pro and show the position where Pro starts.

SQL Query:

```
SELECT  ProdID,  
        ProdName,  
        LOCATE('Pro', ProdName)  
AS ProPosition FROM  
Product  
WHERE LOCATE('Pro',  
ProdName) > 0;
```

Function Used: LOCATE()

Description: LOCATE() returns the starting position of the specified text. If the text is not found, it returns 0.

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT ProdID, ProdName, LOCATE('Pro', ProdName) AS ProPosition FROM Product WHERE LOCATE('Pro', ProdName) > 0;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	ProdID	ProdName	ProPosition
☐ Edit Copy Delete	101	Laptop Pro 15	8

↑ ☐ Check all With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query

Task A12 — Extract Customer First Name

Objective:

Display only the first name of every customer.

SQL Query:

```
SELECT  
    CustID,  
    SUBSTRING(  
        TRIM(CustName),  
        1,  
        LOCATE(' ',  
        TRIM(CustName)) - 1  
    ) AS FirstName FROM Customer;
```

Functions Used: TRIM(), LOCATE(), SUBSTRING()

Description: First, TRIM() removes extra spaces. Then LOCATE() finds the first space, and SUBSTRING() extracts the text before that space.

✓ Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)

-- Task A12 `SELECT CustID, SUBSTRING( TRIM(CustName), 1, LOCATE(' ', TRIM(CustName)) - 1 ) AS FirstName FROM Customer;`

[Edit inline] [Edit] [Create PHP code]

☐ Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

				CustID	FirstName
☐	Edit	Copy	Delete	1	Ali
☐	Edit	Copy	Delete	2	Sara
☐	Edit	Copy	Delete	3	HAMZA
☐	Edit	Copy	Delete	4	Ayesha
☐	Edit	Copy	Delete	5	bilal
☐	Edit	Copy	Delete	6	Fatima
☐	Edit	Copy	Delete	7	Usman
☐	Edit	Copy	Delete	8	Maira

Conclusion

In this lab, different MySQL string functions were applied to customer and product data. The queries demonstrated how to remove unnecessary spaces, change letter cases, join text, extract usernames and domains, replace characters, mask phone numbers, pad IDs, and locate specific text within strings.

THE END